

Portfolio

Projekt: Data-Mart-Erstellung in SQL
1.2. Erarbeitungs-/ Reflexionsphase

Katharina Bloeck

Tabelle language erstellen

```
CREATE TABLE "language" (  
  "language_id" INTEGER,  
  "language" TEXT,  
  PRIMARY KEY("language_id")  
);
```

```
INSERT INTO language(language) VALUES ("deutsch");  
INSERT INTO language(language) VALUES ("englisch");  
INSERT INTO language(language) VALUES ("spanisch");  
INSERT INTO language(language) VALUES ("französisch");  
INSERT INTO language(language) VALUES ("hindi");  
INSERT INTO language(language) VALUES ("russisch");  
INSERT INTO language(language) VALUES ("mandarin");  
INSERT INTO language(language) VALUES ("italienisch");  
INSERT INTO language(language) VALUES ("türkisch");  
INSERT INTO language(language) VALUES ("portugiesisch");
```

Testfall Tabelle language

Überprüfung der Beziehung zwischen den Tabellen book und language anhand der zugeordneten Sprachen eines Buches.

```
SELECT book.title, language.language
FROM language JOIN book
ON language.language_id = book.language_id;
```

	title	language
1	The Shining	englisch
2	Mord im Orientexpress	deutsch
3	Harry Potter und der Stein der Weisen	deutsch
4	1984	englisch
5	Foundation	englisch
6	Il Codice Da Vinci	italienisch
7	Der Hobbit	deutsch
8	Sapiens	englisch
9	Becoming	englisch
10	Dune	englisch

Tabelle genre erstellen

```
CREATE TABLE "genre" (  
  "genre_id" INTEGER,  
  "genre_name" TEXT NOT NULL,  
  PRIMARY KEY("genre_id")  
);
```

```
INSERT INTO genre(genre_name) VALUES ("Krimi");  
INSERT INTO genre(genre_name) VALUES ("Thriller");  
INSERT INTO genre(genre_name) VALUES ("Roman");  
INSERT INTO genre(genre_name) VALUES ("Science Fiction");  
INSERT INTO genre(genre_name) VALUES ("Fantasy");  
INSERT INTO genre(genre_name) VALUES ("Horror");  
INSERT INTO genre(genre_name) VALUES ("Sachbuch");  
INSERT INTO genre(genre_name) VALUES ("Ratgeber");  
INSERT INTO genre(genre_name) VALUES ("Fachbuch");  
INSERT INTO genre(genre_name) VALUES ("Biografie");
```

Tabelle bookGenre erstellen

```
CREATE TABLE "bookGenre" (  
  "book_id" INTEGER,  
  "genre_id" INTEGER,  
  PRIMARY KEY("book_id","genre_id"),  
  FOREIGN KEY("book_id") REFERENCES "book"("book_id"),  
  FOREIGN KEY("genre_id") REFERENCES "genre"("genre_id")  
);
```

```
INSERT INTO bookgenre (book_id, genre_id) VALUES (1, 6);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (1, 2);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (2, 1);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (3, 5);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (3, 3);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (4, 4);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (4, 3);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (5, 4);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (6, 2);
```

Tabelle bookGenre erstellen

```
INSERT INTO bookgenre (book_id, genre_id) VALUES (6, 1);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (7, 5);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (7, 3);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (8, 7);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (9, 10);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (10, 4);  
INSERT INTO bookgenre (book_id, genre_id) VALUES (10, 3);
```

Testfall Tabelle genre / bookGenre

Überprüfung der Zuordnung von Büchern zum Genre über die Zwischentabelle bookGenre am Beispiel „Roman“.

```
SELECT b.title, g.genre_name
FROM bookGenre bg JOIN book b
  ON bg.book_id = b.book_id
JOIN genre g
  ON bg.genre_id = g.genre_id
WHERE g.genre_name = "Roman"
ORDER BY b.title;
```

	title	genre_name
1	1984	Roman
2	Der Hobbit	Roman
3	Dune	Roman
4	Harry Potter und der Stein der Weisen	Roman

Tabelle publisher erstellen

```
-- publisher
CREATE TABLE "publisher" (
  "publisher_id" INTEGER,
  "name" TEXT,
  "country" TEXT,
  PRIMARY KEY("publisher_id")
);

INSERT INTO publisher (name, country) VALUES ('Penguin Books', 'Großbritannien');
INSERT INTO publisher (name, country) VALUES ('HarperCollins', 'USA');
INSERT INTO publisher (name, country) VALUES ('Bloomsbury', 'Großbritannien');
INSERT INTO publisher (name, country) VALUES ('Fischer Verlag', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('Heyne Verlag', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('Bastei Lübbe', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('Klett-Cotta', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('C.H. Beck', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('Goldmann Verlag', 'Deutschland');
INSERT INTO publisher (name, country) VALUES ('Diogenes Verlag', 'Schweiz');
```


Testfall Tabelle publisher

Anzeige aller Verlage mit
den vorhandenen Büchern
zur Überprüfung der
Verlagszuordnung

```
SELECT p.name, p.country,  
b.title, b.isbn  
FROM publisher p JOIN book b  
ON p.publisher_id = b.publisher_id;
```

	name	country	title	isbn
1	Penguin Books	Großbritannien	The Shining	9780000000001
2	HarperCollins	USA	Mord im Orientexpress	9780000000002
3	Bloomsbury	Großbritannien	Harry Potter und der Stein der Weisen	9780000000003
4	Fischer Verlag	Deutschland	1984	9780000000004
5	Heyne Verlag	Deutschland	Foundation	9780000000005
6	Bastei Lübbe	Deutschland	Il Codice Da Vinci	9780000000006
7	Klett-Cotta	Deutschland	Der Hobbit	9780000000007
8	C.H. Beck	Deutschland	Sapiens	9780000000008
9	Goldmann Verlag	Deutschland	Becoming	9780000000009
10	Diogenes Verlag	Schweiz	Dune	9780000000010

Tabelle book erstellen

```
CREATE TABLE "book" (  
  "book_id" INTEGER,  
  "title" TEXT NOT NULL,  
  "year" INTEGER,  
  "publisher_id" INTEGER,  
  "language_id" INTEGER,  
  "isbn" INTEGER NOT NULL,  
  PRIMARY KEY("book_id"),  
  FOREIGN KEY("publisher_id") REFERENCES "publisher"("publisher_id")  
);  
  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('The Shining', 1977, 1, 2, 9780000000001);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Mord im Orientexpress', 1934, 2, 1, 9780000000002);  
INSERT INTO book (title, year, publisher_id, language_id, isbn)  
  VALUES ('Harry Potter und der Stein der Weisen', 1997, 3, 1, 9780000000003);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('1984', 1949, 4, 2, 9780000000004);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Foundation', 1951, 5, 2, 9780000000005);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Il Codice Da Vinci', 2003, 6, 8, 9780000000006);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Der Hobbit', 1937, 7, 1, 9780000000007);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Sapiens', 2011, 8, 2, 9780000000008);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Becoming', 2018, 9, 2, 9780000000009);  
INSERT INTO book (title, year, publisher_id, language_id, isbn) VALUES ('Dune', 1965, 10, 2, 9780000000010);
```

Testfall Tabelle book

Prüfung der Zuordnung
von Buchexemplaren zu
Büchern und deren
Besitzer:innen.

```
SELECT b.title, b.isbn,  
c.condition,  
u.first_name, u.last_name  
FROM book b JOIN copy c  
    ON b.book_id = c.book_id  
JOIN user u  
    ON c.owner_user_id = u.user_id;
```

	title	isbn	condition	first_name	last_name
1	The Shining	9780000000001	NEW	Max	Mustermann
2	The Shining	9780000000001	VERY_GOOD	Eva	Musterfrau
3	Mord im Orientexpress	9780000000002	GOOD	Eva	Musterfrau
4	Harry Potter und der Stein der Weisen	9780000000003	USED	Anna	Schmidt
5	Harry Potter und der Stein der Weisen	9780000000003	NEW	Tom	Meyer
6	1984	9780000000004	DAMAGED	Tom	Meyer
7	1984	9780000000004	VERY_GOOD	Lisa	Fischer
8	Foundation	9780000000005	NEW	Lisa	Fischer
9	Il Codice Da Vinci	9780000000006	VERY_GOOD	Laura	Wagner
10	Der Hobbit	9780000000007	GOOD	Laura	Wagner
11	Sapiens	9780000000008	DAMAGED	Jonas	Becker
12	Becoming	9780000000009	NEW	Lukas	Schulz
13	Dune	9780000000010	VERY_GOOD	Lukas	Schulz

Tabelle author erstellen

```
CREATE TABLE "author" (  
  "author_id" INTEGER,  
  "first_name" TEXT,  
  "last_name" TEXT,  
  PRIMARY KEY("author_id")  
);
```

```
INSERT INTO author (first_name, last_name) VALUES ('Stephen', 'King');  
INSERT INTO author (first_name, last_name) VALUES ('Peter', 'Straub');  
INSERT INTO author (first_name, last_name) VALUES ('Agatha', 'Christie');  
INSERT INTO author (first_name, last_name) VALUES ('J.K.', 'Rowling');  
INSERT INTO author (first_name, last_name) VALUES ('George', 'Orwell');  
INSERT INTO author (first_name, last_name) VALUES ('Isaac', 'Asimov');  
INSERT INTO author (first_name, last_name) VALUES ('Dan', 'Brown');  
INSERT INTO author (first_name, last_name) VALUES ('J.R.R.', 'Tolkien');  
INSERT INTO author (first_name, last_name) VALUES ('Terry', 'Pratchett');  
INSERT INTO author (first_name, last_name) VALUES ('Yuval', 'Harari');  
INSERT INTO author (first_name, last_name) VALUES ('Michelle', 'Obama');  
INSERT INTO author (first_name, last_name) VALUES ('Frank', 'Herbert');
```

Tabelle bookAuthor erstellen

```
CREATE TABLE "bookAuthor" (  
  "book_id" INTEGER,  
  "author_id" INTEGER,  
  PRIMARY KEY("book_id","author_id"),  
  FOREIGN KEY("author_id") REFERENCES "author"("author_id"),  
  FOREIGN KEY("book_id") REFERENCES "book"("book_id")  
);
```

```
INSERT INTO bookauthor (book_id, author_id) VALUES (1, 1);  
INSERT INTO bookauthor (book_id, author_id) VALUES (1, 2);  
INSERT INTO bookauthor (book_id, author_id) VALUES (2, 3);  
INSERT INTO bookauthor (book_id, author_id) VALUES (3, 4);  
INSERT INTO bookauthor (book_id, author_id) VALUES (4, 5);  
INSERT INTO bookauthor (book_id, author_id) VALUES (5, 6);  
INSERT INTO bookauthor (book_id, author_id) VALUES (6, 7);  
INSERT INTO bookauthor (book_id, author_id) VALUES (7, 8);  
INSERT INTO bookauthor (book_id, author_id) VALUES (7, 9);  
INSERT INTO bookauthor (book_id, author_id) VALUES (8, 10);  
INSERT INTO bookauthor (book_id, author_id) VALUES (9, 11);  
INSERT INTO bookauthor (book_id, author_id) VALUES (10, 12);
```

Testfall Tabelle author / bookAuthor

Überprüfung der Zuordnung
zwischen Büchern und
Autor:innen über die Tabelle
bookAuthor.

```
SELECT b.title, a.first_name, a.last_name
FROM bookAuthor ba
JOIN book b
  ON ba.book_id = b.book_id
JOIN author a
  ON ba.author_id = a.author_id;
```

	title	first_name	last_name
1	The Shining	Stephen	King
2	The Shining	Peter	Straub
3	Mord im Orientexpress	Agatha	Christie
4	Harry Potter und der Stein der Weisen	J.K.	Rowling
5	1984	George	Orwell
6	Foundation	Isaac	Asimov
7	Il Codice Da Vinci	Dan	Brown
8	Der Hobbit	J.R.R.	Tolkien
9	Der Hobbit	Terry	Pratchett
10	Sapiens	Yuval	Harari
11	Becoming	Michelle	Obama
12	Dune	Frank	Herbert

Tabelle user erstellen

```
CREATE TABLE "user" (  
  "user_id" INTEGER,  
  "first_name" TEXT NOT NULL,  
  "last_name" TEXT NOT NULL,  
  "email" TEXT NOT NULL,  
  PRIMARY KEY("user_id")  
);  
  
INSERT INTO user (first_name, last_name, email) VALUES ('Max', 'Mustermann', 'mmustermann@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Eva', 'Musterfrau', 'emusterfrau@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Sophie', 'Krause', 'skrause@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Anna', 'Schmidt', 'ananas@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Tom', 'Meyer', 'tomm@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Lisa', 'Fischer', 'lifi@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Paul', 'Weber', 'paulwe@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Laura', 'Wagner', 'lauwa@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Jonas', 'Becker', 'jb@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Nina', 'Hoffmann', 'ninahoma@mail.de');  
INSERT INTO user (first_name, last_name, email) VALUES ('Lukas', 'Schulz', 'lukass@mail.de');
```


Testfall Tabelle user

Anzeige aller User und
Userdaten

```
SELECT * FROM user;
```

	user_id	first_name	last_name	email
1	1	Max	Mustermann	mmustermann@mail.de
2	2	Eva	Musterfrau	emusterfrau@mail.de
3	3	Sophie	Krause	skrause@mail.de
4	4	Anna	Schmidt	ananas@mail.de
5	5	Tom	Meyer	tomm@mail.de
6	6	Lisa	Fischer	liff@mail.de
7	7	Paul	Weber	paulwe@mail.de
8	8	Laura	Wagner	lauwa@mail.de
9	9	Jonas	Becker	jb@mail.de
10	10	Nina	Hoffmann	ninahoma@mail.de
11	11	Lukas	Schulz	lukass@mail.de

Tabelle role erstellen

```
CREATE TABLE "role" (  
  "role_id" INTEGER,  
  "role_name" TEXT NOT NULL,  
  PRIMARY KEY("role_id")  
);
```

```
INSERT INTO role(role_name) VALUES ('Borrower');  
INSERT INTO role(role_name) VALUES ('Owner');
```

Tabelle userRole erstellen

```
CREATE TABLE "userRole" (  
  "user_id" INTEGER,  
  "role_id" INTEGER,  
  PRIMARY KEY("user_id", "role_id"),  
  FOREIGN KEY("role_id") REFERENCES "role"("role_id"),  
  FOREIGN KEY("user_id") REFERENCES "user"("user_id")  
);
```

```
INSERT INTO userRole VALUES(1, 1);  
INSERT INTO userRole VALUES(1, 2);  
INSERT INTO userRole VALUES(2, 2);  
INSERT INTO userRole VALUES(3, 1);  
INSERT INTO userRole VALUES(4, 2);  
INSERT INTO userRole VALUES(5, 2);  
INSERT INTO userRole VALUES(6, 1);  
INSERT INTO userRole VALUES(6, 2);  
INSERT INTO userRole VALUES(7, 1);  
INSERT INTO userRole VALUES(8, 2);  
INSERT INTO userRole VALUES(9, 2);  
INSERT INTO userRole VALUES(10, 1);  
INSERT INTO userRole VALUES(11, 2);
```

Testfall Tabelle user / role / userRole

Überprüfung der korrekten Zuordnung von Benutzer:innen zu ihren Rollen über die Zwischentabelle userRole.

```
SELECT user.user_id, user.first_name, user.last_name,  
role.role_name  
FROM user  
JOIN userRole  
    ON user.user_id = userRole.user_id  
JOIN role  
    ON userRole.role_id = role.role_id;
```

	user_id	first_name	last_name	role_name
1	1	Max	Mustermann	Borrower
2	1	Max	Mustermann	Owner
3	2	Eva	Musterfrau	Owner
4	3	Sophie	Krause	Borrower
5	4	Anna	Schmidt	Owner
6	5	Tom	Meyer	Owner
7	6	Lisa	Fischer	Borrower
8	6	Lisa	Fischer	Owner
9	7	Paul	Weber	Borrower
10	8	Laura	Wagner	Owner
11	9	Jonas	Becker	Owner
12	10	Nina	Hoffmann	Borrower
13	11	Lukas	Schulz	Owner

Tabelle copy erstellen

```
CREATE TABLE "copy" (  
  "copy_id" INTEGER,  
  "book_id" INTEGER,  
  "owner_user_id" INTEGER,  
  "condition" TEXT CHECK("condition" IN ('NEW', 'VERY_GOOD', 'GOOD', 'USED', 'DAMAGED')),  
  PRIMARY KEY("copy_id"),  
  FOREIGN KEY("book_id") REFERENCES "book"("book_id"),  
  FOREIGN KEY("owner_user_id") REFERENCES "user"("user_id")  
);
```

```
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (1, 1, "NEW");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (1, 2, "VERY_GOOD");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (2, 2, "GOOD");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (3, 4, "USED");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (3, 5, "NEW");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (4, 5, "DAMAGED");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (4, 6, "VERY_GOOD");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (5, 6, "NEW");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (6, 8, "VERY_GOOD");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (7, 8, "GOOD");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (8, 9, "DAMAGED");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (9, 11, "NEW");  
INSERT INTO copy(book_id, owner_user_id, condition) VALUES (10, 11, "VERY_GOOD");
```

Testfall Tabelle copy

Überprüfung der Zuordnung
von Buchexemplaren zu
Büchern und deren
Besitzer:innen.

```
SELECT b.title, c.condition, u.email
FROM copy c
JOIN book b
  ON c.book_id = b.book_id
JOIN user u
  ON c.owner_user_id = u.user_id;
```

	title	condition	email
1	The Shining	NEW	mmustermann@mail.de
2	The Shining	VERY_GOOD	emusterfrau@mail.de
3	Mord im Orientexpress	GOOD	emusterfrau@mail.de
4	Harry Potter und der Stein der Weisen	USED	ananas@mail.de
5	Harry Potter und der Stein der Weisen	NEW	tomm@mail.de
6	1984	DAMAGED	tomm@mail.de
7	1984	VERY_GOOD	lifi@mail.de
8	Foundation	NEW	lifi@mail.de
9	Il Codice Da Vinci	VERY_GOOD	lauwa@mail.de
10	Der Hobbit	GOOD	lauwa@mail.de
11	Sapiens	DAMAGED	jb@mail.de
12	Becoming	NEW	lukass@mail.de
13	Dune	VERY_GOOD	lukass@mail.de

Tabelle location erstellen

```
CREATE TABLE "location" (  
  "location_id" INTEGER,  
  "user_id" INTEGER,  
  "location_type" TEXT NOT NULL CHECK("location_type" IN ('PICKUP', 'MEETUP', 'SHIPPING')),  
  "street" TEXT NOT NULL,  
  "number" INTEGER NOT NULL,  
  "postal_code" INTEGER NOT NULL,  
  "city" TEXT NOT NULL,  
  PRIMARY KEY("location_id"),  
  FOREIGN KEY("user_id") REFERENCES "user"("user_id")  
);
```

```
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (1, 'PICKUP', 'Musterstraße', 12, 10115, 'Berlin');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (1, 'SHIPPING', 'Musterstraße', 12, 10115, 'Berlin');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (2, 'MEETUP', 'Hauptstraße', 45, 20095, 'Hamburg');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (3, 'SHIPPING', 'Baumallee', 2, 30165, 'Hannover');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (4, 'PICKUP', 'Bahnhofstraße', 3, 80331, 'München');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (5, 'MEETUP', 'Lindenweg', 27, 50667, 'Köln');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (6, 'PICKUP', 'Goethestraße', 8, 60313, 'Frankfurt');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (7, 'SHIPPING', 'Versandweg', 10, 12345, 'Leipzig');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (8, 'MEETUP', 'Schillerstraße', 19, 70173, 'Stuttgart');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (9, 'PICKUP', 'Rosenstraße', 5, 40213, 'Düsseldorf');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (9, 'SHIPPING', 'Rosenstraße', 5, 40213, 'Düsseldorf');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (10, 'SHIPPING', 'Poststraße', 22, 90402, 'Nürnberg');  
INSERT INTO location (user_id, location_type, street, number, postal_code, city) VALUES (11, 'MEETUP', 'Parkallee', 33, 28195, 'Bremen');
```

Testfall Tabelle location

Überprüfung der Zuordnung von Benutzer:innen zu ihren hinterlegten Standorten.

```
SELECT
    u.first_name || ' ' || u.last_name AS user,
    l.location_type,
    l.city
FROM location l
JOIN user u
    ON l.user_id = u.user_id
ORDER BY l.city;
```

	user	location_type	city
1	Max Mustermann	PICKUP	Berlin
2	Max Mustermann	SHIPPING	Berlin
3	Max Mustermann	PICKUP	Berlin
4	Lukas Schulz	MEETUP	Bremen
5	Jonas Becker	PICKUP	Düsseldorf
6	Jonas Becker	SHIPPING	Düsseldorf
7	Lisa Fischer	PICKUP	Frankfurt
8	Eva Musterfrau	MEETUP	Hamburg
9	Sophie Krause	SHIPPING	Hannover
10	Tom Meyer	MEETUP	Köln
11	Paul Weber	SHIPPING	Leipzig
12	Anna Schmidt	PICKUP	München
13	Nina Hoffmann	SHIPPING	Nürnberg
14	Laura Wagner	MEETUP	Stuttgart

Tabelle offer erstellen

```
CREATE TABLE "offer" (  
  "offer_id" INTEGER,  
  "copy_id" INTEGER,  
  "location_id" INTEGER,  
  "max_days" INTEGER CHECK("max_days" > 0),  
  "shipping_possible" INTEGER NOT NULL CHECK("shipping_possible" IN (0, 1)),  
  "is_active" INTEGER NOT NULL CHECK("is_active" IN (0, 1)),  
  PRIMARY KEY("offer_id"),  
  FOREIGN KEY("copy_id") REFERENCES "copy"("copy_id"),  
  FOREIGN KEY("location_id") REFERENCES "location"("location_id")  
);  
  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (1, 1, 14, 1, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (2, 3, 21, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (3, 3, 14, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (4, 5, 10, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (5, 6, 14, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (6, 6, 7, 0, 0);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (7, 7, 21, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (8, 7, 30, 1, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (9, 9, 14, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (10, 9, 21, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (11, 10, 7, 1, 0);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (12, 13, 14, 0, 1);  
INSERT INTO offer (copy_id, location_id, max_days, shipping_possible, is_active) VALUES (13, 13, 30, 0, 1);
```

Testfall Tabelle offer

Anzeige aller Bücher, die per Versand ausgeliehen werden können

	title	isbn	condition
1	The Shining	9780000000001	NEW
2	Foundation	9780000000005	NEW
3	Sapiens	9780000000008	DAMAGED
4	The Shining	9780000000001	NEW

```
SELECT b.title, b.isbn, c.condition
FROM offer o
JOIN copy c
    ON o.copy_id = c.copy_id
JOIN book b
    ON b.book_id = c.book_id
WHERE shipping_possible = 1;
```

Tabelle transaction erstellen

```
CREATE TABLE "transactions" (  
  "transaction_id" INTEGER,  
  "borrower_user_id" INTEGER,  
  "offer_id" INTEGER,  
  "status" TEXT NOT NULL CHECK(("status" IN ('REQUESTED', 'APPROVED', 'ACTIVE', 'RETURNED', 'CANCELLED'))),  
  "requested_at" TEXT,  
  "approved_at" TEXT,  
  "due_at" TEXT,  
  "returned_at" TEXT,  
  PRIMARY KEY("transaction_id"),  
  FOREIGN KEY("borrower_user_id") REFERENCES "user"("user_id"),  
  FOREIGN KEY("offer_id") REFERENCES "offer"("offer_id")  
);
```

Tabelle transaction erstellen

```
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (2, 1, 'RETURNED', '2026-04-01 10:00:00', '2026-04-01 12:00:00', '2026-04-15 12:00:00', '2026-04-14 18:00:00');
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (1, 2, 'ACTIVE', '2026-04-10 09:00:00', '2026-04-10 10:00:00', '2026-04-24 10:00:00', NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (5, 3, 'APPROVED', '2026-04-15 14:00:00', '2026-04-15 16:00:00', '2026-04-29 16:00:00', NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (6, 4, 'REQUESTED', '2026-04-20 11:00:00', NULL, NULL, NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (4, 5, 'RETURNED', '2026-03-10 08:00:00', '2026-03-10 09:00:00', '2026-03-20 09:00:00', '2026-03-19 17:00:00');
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (8, 6, 'CANCELLED', '2026-04-05 13:00:00', NULL, NULL, NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (9, 7, 'ACTIVE', '2026-04-18 15:00:00', '2026-04-18 16:00:00', '2026-05-02 16:00:00', NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (11, 8, 'RETURNED', '2026-03-01 10:00:00', '2026-03-01 12:00:00', '2026-03-15 12:00:00', '2026-03-14 19:00:00');
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (2, 9, 'APPROVED', '2026-04-22 17:00:00', '2026-04-22 18:00:00', '2026-05-06 18:00:00', NULL);
INSERT INTO "transactions" (borrower_user_id, offer_id, status, requested_at, approved_at, due_at, returned_at)
VALUES (6, 10, 'REQUESTED', '2026-04-25 09:30:00', NULL, NULL, NULL);
```

Testfall Tabelle transaction

Überprüfung der Dokumentation von Ausleihvorgängen einschließlich ausleihender Person, Status- und Datumsinformationen.

```
SELECT  
u.last_name AS borrower,  
b.title,  
t.status,  
DATE(t.requested_at) AS requested_at,  
DATE(t.due_at) AS due_at,  
DATE(t.returned_at) AS returned_at  
FROM transactions t  
JOIN user u  
    ON t.borrower_user_id = u.user_id  
JOIN offer o  
    ON t.offer_id = o.offer_id  
JOIN copy c  
    ON o.copy_id = c.copy_id  
JOIN book b  
    ON c.book_id = b.book_id;
```

	borrower	title	status	requested_at	due_at	returned_at
1	Musterfrau	The Shining	RETURNED	2026-04-01	2026-04-15	2026-04-14
2	Mustermann	The Shining	ACTIVE	2026-04-10	2026-04-24	NULL
3	Meyer	Mord im Orientexpress	APPROVED	2026-04-15	2026-04-29	NULL
4	Fischer	Harry Potter und der Stein...	REQUESTED	2026-04-20	NULL	NULL
5	Schmidt	Harry Potter und der Stein...	RETURNED	2026-03-10	2026-03-20	2026-03-19
6	Wagner	1984	CANCELLED	2026-04-05	NULL	NULL
7	Becker	1984	ACTIVE	2026-04-18	2026-05-02	NULL
8	Schulz	Foundation	RETURNED	2026-03-01	2026-03-15	2026-03-14
9	Musterfrau	Il Codice Da Vinci	APPROVED	2026-04-22	2026-05-06	NULL
10	Fischer	Der Hobbit	REQUESTED	2026-04-25	NULL	NULL
11	Musterfrau	The Shining	RETURNED	2026-04-01	2026-04-15	2026-04-14

Tabelle review erstellen

```
CREATE TABLE "review" (  
  "review_id" INTEGER,  
  "reviewer_user_id" INTEGER,  
  "book_id" INTEGER,  
  "rating" INTEGER NOT NULL CHECK("rating" BETWEEN 1 AND 5),  
  "comment" TEXT,  
  PRIMARY KEY("review_id"),  
  FOREIGN KEY("book_id") REFERENCES "book"("book_id"),  
  FOREIGN KEY("reviewer_user_id") REFERENCES "user"("user_id")  
);
```

```
INSERT INTO review (reviewer_user_id, book_id, rating, comment) VALUES (1, 1, 5, 'Spannend von Anfang bis Ende, absolut empfehlenswert!');  
INSERT INTO review (reviewer_user_id, book_id, rating, comment) VALUES (2, 2, 4, 'Klassischer Krimi, sehr gut geschrieben.');
```

reviewer_user_id	book_id	rating	comment
1	1	5	Spannend von Anfang bis Ende, absolut empfehlenswert!
2	2	4	Klassischer Krimi, sehr gut geschrieben.
4	3	5	Magische Welt, hat mich komplett gefesselt.
5	4	5	
6	5	4	Interessante Ideen, teilweise etwas komplex.
8	6	3	Unterhaltsam, aber etwas vorhersehbar.
9	7	5	
11	8	5	Sehr informativ und gut verständlich.
2	9	4	Inspirierende Geschichte.
6	10	5	Episch und tiefgründig, eines meiner Lieblingsbücher.
5	1	2	Nicht ganz mein Geschmack, aber solide geschrieben.
8	3	4	Sehr unterhaltsam, würde ich weiterempfehlen.

```
INSERT INTO review (reviewer_user_id, book_id, rating, comment) VALUES (4, 3, 5, 'Magische Welt, hat mich komplett gefesselt.');
```

Testfall Tabelle review

Überprüfung der
Zuordnung von
Bewertungen und
Kommentaren zu
Büchern

```
SELECT b.title, r.rating, r.comment
FROM review r
JOIN book b
  ON r.book_id = b.book_id
ORDER BY rating;
```

	title	rating	comment
1	The Shining	2	Nicht ganz mein Geschmack, aber solide geschrieben.
2	Il Codice Da Vinci	3	Unterhaltsam, aber etwas vorhersehbar.
3	Mord im Orientexpress	4	Klassischer Krimi, sehr gut geschrieben.
4	Foundation	4	Interessante Ideen, teilweise etwas komplex.
5	Becoming	4	Inspirierende Geschichte.
6	Harry Potter und der Stein der Weisen	4	Sehr unterhaltsam, würde ich weiterempfehlen.
7	The Shining	5	Spannend von Anfang bis Ende, absolut empfehlenswert!
8	Harry Potter und der Stein der Weisen	5	Magische Welt, hat mich komplett gefesselt.
9	1984	5	<i>NULL</i>
10	Der Hobbit	5	<i>NULL</i>
11	Sapiens	5	Sehr informativ und gut verständlich.
12	Dune	5	Episch und tiefgründig, eines meiner Lieblingsbücher.

Indizes anlegen

Anlegen von Indizes zur Verbesserung der Performance für häufig genutzte Fremdschlüssel

```
-- Anlegen von Indizes
CREATE INDEX idx_copy_book_id ON copy(book_id);
CREATE INDEX idx_copy_owner_user_id ON copy(owner_user_id);
CREATE INDEX idx_offer_copy_id ON offer(copy_id);
CREATE INDEX idx_transactions_offer_id ON transactions(offer_id);
CREATE INDEX idx_transactions_borrower_user_id ON transactions(borrower_user_id);
CREATE INDEX idx_review_book_id ON review(book_id);
CREATE INDEX idx_bookgenre_genre_id ON bookGenre(genre_id);
CREATE INDEX idx_bookauthor_author_id ON bookAuthor(author_id);
```